Министерство науки и высшего образования Российской Федерации
Московский государственный технический университет имени Н Э Баумана
Факультет Информатика и системы управления
Кафедра Системы обработки информации и управления

Лабораторная работа 5
Разработка простого бота для Telegram

Дисциплина Парадигмы и конструкции языков программирования

Выполнил: студент группы ИУ5-32Б
Трефилов Д.И.
Проверил: _____
Дата защиты: _____

Москва 2026

1 Цель работы

Освоить команды Telegram-бота, inline-кнопки и обработку callback-запросов.

2 Постановка задачи

Разработать простого Telegram-бота с функциональностью создания кнопок. Предметная область - справочник площадей геометрических фигур.

Источник задания: https://github.com/ugapanyuk/BKIT_2021/wiki/lab_bot1

Список курса: https://github.com/ugapanyuk/courses_content/wiki/COURSE_PCPL_MAIN

3 Структура и логика программы

Команда /start выводит InlineKeyboardMarkup с кнопками Квадрат, Прямоугольник и Круг. Нажатие отправляет callback_data. choose_figure подтверждает callback вызовом answer и редактирует сообщение, показывая формулу и пример. Клавиатура сохраняется для повторного выбора. /help выводит инструкцию. Токен читается из TELEGRAM_BOT_TOKEN; при его отсутствии программа завершает запуск с понятным сообщением. Обычный запуск main.py открывает ручной ввод: пользователь выбирает пункт меню, вводит свои данные и получает результат. После проверки можно ввести другой набор. Готовые примеры запускаются с --demo. В ЛР5 и ЛР6 подключение к Telegram запускается отдельно с --telegram; по умолчанию используется локальный диалог без токена. Команда /area позволяет ввести свои размеры и в Telegram.

4 Порядок запуска

Команды выполняются из папки работы в подготовленном окружении.

```
python -m pip install -r requirements.txt
python main.py
python -m unittest discover -s tests -v
```

Запуск с подключением к Telegram в PowerShell:

```
$env:TELEGRAM_BOT_TOKEN = 'ваш_токен'
python main.py --telegram
```

Файл .env автоматически не загружается. Токен в отчет и Git не добавляется.

5 Проверка программы

Пять асинхронных теста проверяют кнопки /start, все три формулы, неизвестный callback и регистрацию обработчиков. AsyncMock подменяет ответы Telegram API. main.py --demo вызывает настоящие обработчики с локальным выводом. Доступ к Telegram API и реальный диалог в Telegram не проверялись: токен пользователя не предоставлен. Экранная форма ниже показывает локальную демонстрацию, а не интерфейс Telegram. Дополнительный тест проверяет расчет пользовательских размеров и ошибки команды /area.

Пример выполнения

```
$ python main.py --demo
Локальная демонстрация обработчиков, без подключения к Telegram.
Пользователь: /start
Бот: Справочник площадей. Выберите фигуру:
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь нажал: square
Бот: Квадрат
S = a². При a = 5: S = 25.
Свои размеры: /area square 5
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь нажал: rectangle
Бот: Прямоугольник
S = a × b. При a = 5, b = 4: S = 20.
Свои размеры: /area rectangle 5 4
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь нажал: circle
Бот: Круг
S = π × r². При r = 5: S ≈ 78.54.
Свои размеры: /area circle 5
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь: /help
Бот: /start — открыть кнопки
/help — справка
Нажмите кнопку, чтобы узнать формулу и пример расчета.
Свои размеры: /area square 5, /area rectangle 5 4 или /area circle 5
```

Экранная форма выполнения


```
$ python main.py --demo
Локальная демонстрация обработчиков, без подключения к Telegram.
Пользователь: /start
Бот: Справочник площадей. Выберите фигуру:
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь нажал: square
Бот: Квадрат
S = a². При a = 5: S = 25.
Свои размеры: /area square 5
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь нажал: rectangle
Бот: Прямоугольник
S = a × b. При a = 5, b = 4: S = 20.
Свои размеры: /area rectangle 5 4
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь нажал: circle
Бот: Круг
S = π × r². При r = 5: S ≈ 78.54.
Свои размеры: /area circle 5
Кнопки: Квадрат, Прямоугольник, Круг
Пользователь: /help
Бот: /start — открыть кнопки
/help — справка
Нажмите кнопку, чтобы узнать формулу и пример расчета.
Свои размеры: /area square 5, /area rectangle 5 4 или /area circle 5
```

Ручная проверка со своими значениями

```
$ python main.py
Ввод по строкам: 2 | 6 7 | 0
Локальный режим без токена Telegram. Вводите номера кнопок.
Бот: Справочник площадей. Выберите фигуру:
Кнопки: Квадрат, Прямоугольник, Круг

1 — Квадрат; 2 — Прямоугольник; 3 — Круг; 0 — Выход: Бот: Прямоугольник
S = a × b. При a = 5, b = 4: S = 20.
Свои размеры: /area rectangle 5 4
Кнопки: Квадрат, Прямоугольник, Круг
Две стороны через пробел: Бот: Площадь: 42.00

1 — Квадрат; 2 — Прямоугольник; 3 — Круг; 0 — Выход:
```

Модульные тесты

```
test_all_buttons (test_bot.TestBot.test_all_buttons) ... ok
test_application_configuration (test_bot.TestBot.test_application_configuration) ...
ok
test_custom_dimensions (test_bot.TestBot.test_custom_dimensions) ... ok
test_start_buttons (test_bot.TestBot.test_start_buttons) ... ok
test_unknown_callback (test_bot.TestBot.test_unknown_callback) ... ok

----------------------------------------------------------------------
Ran 5 tests in 3.595s

OK
```


```
test_all_buttons (test_bot.TestBot.test_all_buttons) ... ok
test_application_configuration (test_bot.TestBot.test_application_configuration) ...
ok
test_custom_dimensions (test_bot.TestBot.test_custom_dimensions) ... ok
test_start_buttons (test_bot.TestBot.test_start_buttons) ... ok
test_unknown_callback (test_bot.TestBot.test_unknown_callback) ... ok

----------------------------------------------------------------------
Ran 5 tests in 3.595s

OK
```


6 Выводы

Бот реализует команды и inline-кнопки; локальные тесты подтверждают ответы обработчиков. Для проверки в Telegram необходимо задать личный токен и запустить polling.

Приложение Листинги программ

demo.py

```
"""Локальный вызов настоящих обработчиков без Telegram API и токена."""
import asyncio
from types import SimpleNamespace
from lab_telegram_bot.bot import start, choose_figure, help_command


async def display(text, **kwargs):
    print("Бот:", text)
    markup = kwargs.get("reply_markup")
    if markup:
        print("Кнопки:", ", ".join(b.text for row in markup.inline_keyboard for b in row))


async def answer():
    pass


async def demo():
    print("Локальная демонстрация обработчиков, без подключения к Telegram.")
    message = SimpleNamespace(reply_text=display)
    query = SimpleNamespace(data="square", answer=answer, edit_message_text=display)
    update = SimpleNamespace(effective_message=message, callback_query=query)
    print("Пользователь: /start")
    await start(update, None)
    for key in ("square", "rectangle", "circle"):
        print("Пользователь нажал:", key)
        query.data = key
        await choose_figure(update, None)
    print("Пользователь: /help")
    await help_command(update, None)


def run():
    asyncio.run(demo())


if __name__ == "__main__":
    run()
```

interactive.py

```
"""Ручной вызов тех же обработчиков, которые использует Telegram."""
import asyncio
from types import SimpleNamespace
from demo import display, answer
from lab_telegram_bot.bot import start, choose_figure, area_command


async def interactive():
    print("Локальный режим без токена Telegram. Вводите номера кнопок.")
    query = SimpleNamespace(data="", answer=answer, edit_message_text=display)
    update = SimpleNamespace(effective_message=SimpleNamespace(reply_text=display),
                             callback_query=query)
    context = SimpleNamespace(args=[])
    await start(update, context)
    while True:
        choice = input("\n1 — Квадрат; 2 — Прямоугольник; 3 — Круг; 0 — Выход: ").strip()
        if choice == "0":
            return
        shape = {"1": "square", "2": "rectangle", "3": "circle"}.get(choice)
        if shape is None:
            print("Введите номер от 0 до 3.")
            continue
        query.data = shape
        await choose_figure(update, context)
        text = input("Две стороны через пробел: " if shape == "rectangle" else
                     "Радиус: " if shape == "circle" else "Сторона: ")
        context.args = [shape, *text.split()]
        await area_command(update, context)


def run():
    asyncio.run(interactive())
```

lab_telegram_bot/bot.py

```
"""Справочник геометрических фигур с inline-клавиатурой."""
import os
from telegram import InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, CallbackQueryHandler
from .calculator import calculate

FIGURES = {
    "square": ("Квадрат", "S = a². При a = 5: S = 25."),
    "rectangle": ("Прямоугольник", "S = a × b. При a = 5, b = 4: S = 20."),
    "circle": ("Круг", "S = π × r². При r = 5: S ≈ 78.54."),
}


def keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(title, callback_data=key)]
        for key, (title, _) in FIGURES.items()
    ])


async def start(update, context):
    await update.effective_message.reply_text(
        "Справочник площадей. Выберите фигуру:", reply_markup=keyboard()
    )


async def help_command(update, context):
    await update.effective_message.reply_text(
        "/start — открыть кнопки\n/help — справка\n"
        "Нажмите кнопку, чтобы узнать формулу и пример расчета.\n"
        "Свои размеры: /area square 5, /area rectangle 5 4 или /area circle 5"
    )


async def choose_figure(update, context):
    query = update.callback_query
    await query.answer()
    figure = FIGURES.get(query.data)
    text = "Неизвестная фигура. Выберите кнопку ниже."
    if figure is not None:
        text = f"{figure[0]}\n{figure[1]}\nСвои размеры: /area {query.data} " + (
            "5 4" if query.data == "rectangle" else "5")
    await query.edit_message_text(text, reply_markup=keyboard())


async def area_command(update, context):
    try:
        if not context.args:
            raise ValueError("Пример: /area rectangle 5 4")
        area = calculate(context.args[0].casefold(), " ".join(context.args[1:]))
        text = f"Площадь: {area:.2f}"
    except ValueError as exc:
        text = f"Ошибка: {exc}"
    await update.effective_message.reply_text(text)


def build_application(token):
    app = Application.builder().token(token).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("help", help_command))
    app.add_handler(CommandHandler("area", area_command))
    app.add_handler(CallbackQueryHandler(choose_figure))
    return app


def main():
```

```
    token = os.environ.get("TELEGRAM_BOT_TOKEN")
    if not token:
        raise SystemExit("Задайте переменную окружения TELEGRAM_BOT_TOKEN из BotFather.")
    build_application(token).run_polling()
```

lab_telegram_bot/calculator.py

```
"""Общий расчет для команды Telegram и ручного ввода в консоли."""
import math


def calculate(shape, text):
    if shape not in ("square", "rectangle", "circle"):
        raise ValueError("Фигура: square, rectangle или circle")
    values = [float(value.replace(",", ".")) for value in text.split()]
    count = 2 if shape == "rectangle" else 1
    if len(values) != count:
        raise ValueError(f"Введите размеров: {count}")
    if any(not math.isfinite(x) or x <= 0 for x in values):
        raise ValueError("Нужны конечные положительные размеры")
    try:
        area = values[0] * values[1] if shape == "rectangle" else values[0] ** 2
        if shape == "circle":
            area *= math.pi
    except OverflowError as exc:
        raise ValueError("Размеры слишком велики") from exc
    if not math.isfinite(area) or area <= 0:
        raise ValueError("Размеры вне поддерживаемого диапазона")
    return area
```

main.py

```
import sys
from lab_telegram_bot.bot import main


if __name__ == "__main__":
    try:
        if "--demo" in sys.argv:
            from demo import run
            run()
        elif "--telegram" in sys.argv:
            main()
        else:
            from interactive import run
            run()
    except (EOFError, KeyboardInterrupt):
        print("\nПрограмма завершена.")
```

requirements.txt

```
python-telegram-bot==22.8
```

tests/test_bot.py

```
import unittest
from types import SimpleNamespace
from unittest.mock import AsyncMock
from lab_telegram_bot.bot import start, choose_figure, keyboard, build_application, area_command


class TestBot(unittest.IsolatedAsyncioTestCase):
    async def test_start_buttons(self):
        message = SimpleNamespace(reply_text=AsyncMock())
        await start(SimpleNamespace(effective_message=message), None)
        markup = message.reply_text.call_args.kwargs["reply_markup"]
        self.assertEqual(len(markup.inline_keyboard), 3)

    async def test_all_buttons(self):
        for key, expected in (("square", "25"), ("rectangle", "20"), ("circle", "78.54")):
            with self.subTest(key=key):
                query = SimpleNamespace(data=key, answer=AsyncMock(),
edit_message_text=AsyncMock())
                await choose_figure(SimpleNamespace(callback_query=query), None)
                query.answer.assert_awaited_once()
                self.assertIn(expected, query.edit_message_text.call_args.args[0])

    async def test_unknown_callback(self):
        query = SimpleNamespace(data="bad", answer=AsyncMock(), edit_message_text=AsyncMock())
        await choose_figure(SimpleNamespace(callback_query=query), None)
        self.assertIn("Неизвестная", query.edit_message_text.call_args.args[0])

    async def test_application_configuration(self):
        app = build_application("123456:LOCAL_TEST_TOKEN")
        self.assertEqual(len(app.handlers[0]), 4)
        self.assertEqual([row[0].callback_data for row in keyboard().inline_keyboard],
                         ["square", "rectangle", "circle"])

    async def test_custom_dimensions(self):
        message = SimpleNamespace(reply_text=AsyncMock())
        for args, expected in ((["rectangle", "6", "7"], "42.00"),
                               (["square", "2,5"], "6.25"),
                               (["circle", "2"], "12.57"),
                               (["square", "nan"], "Ошибка"),
                               (["square", "-2"], "Ошибка"),
                               (["rectangle", "2"], "Ошибка"),
                               (["square", "1e308"], "Ошибка"),
                               ([], "Ошибка")):
            with self.subTest(args=args):
                await area_command(SimpleNamespace(effective_message=message),
                                   SimpleNamespace(args=args))
                self.assertIn(expected, message.reply_text.call_args.args[0])


if __name__ == "__main__":
    unittest.main()
```